GUÍA DE ESTUDIO

HTML, CSS y JavaScript

Fundamentos, estructura, variables, funcionalidades y cómo se conectan

Objetivo: comprender cómo se construye una página web, cómo se diseña y cómo se le agrega interactividad. Incluye ejemplos prácticos y un proyecto completo para realizar en Visual Studio Code.

Una idea para recordar durante todo el aprendizaje:

HTML construye · CSS diseña · JavaScript hace que las cosas sucedan.

Material de consulta para principiantes · Ejemplos comentados · Proyecto paso a paso

Cómo utilizar esta guía

No intentes memorizar todos los conceptos en una sola sesión. Leé una sección, probá los ejemplos en el editor y modificá el código para observar qué cambia. El objetivo es entenderlo, no copiarlo sin pensar.

Orden recomendado: HTML → CSS → JavaScript → integración de las tres tecnologías.

1. ¿Cómo funciona el desarrollo web?

Una página web se construye principalmente con tres tecnologías:

Tecnología	Función	Ejemplo
HTML	Estructura y contenido.	Títulos, párrafos, imágenes, enlaces y botones.
CSS	Diseño y presentación visual.	Colores, tipografías, espacios y distribución.
JavaScript	Lógica e interactividad.	Menús desplegables, validaciones y contadores.

Ejemplo: en una página para una cervecería llamada Bruma, HTML crea el título, la descripción y los botones; CSS define la identidad visual y la composición; JavaScript puede controlar un menú interactivo o actualizar el contenido cuando el usuario hace clic.

Estas tecnologías tienen responsabilidades diferentes, pero trabajan juntas en el navegador.

2. HTML: la estructura de la página

HTML significa *HyperText Markup Language*. Es un lenguaje de marcado que organiza el contenido de una página web.

2.1. Estructura básica

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi primera página</title>

    <link rel="stylesheet" href="css/styles.css">
    <script src="js/script.js" defer></script>
</head>
<body>
    <header>
        <h1>Mi primera página web</h1>
    </header>
    <main>
        <p>Estoy aprendiendo desarrollo web.</p>
        <button id="miBoton">Hacé clic</button>
        <p id="mensaje"></p>
    </main>
</body>
</html>
```

Qué significa cada parte:

• **<!DOCTYPE html>**: indica que el documento usa HTML moderno.

• **<html lang="es">**: elemento raíz; el idioma principal es español.

• **<head>**: contiene metadatos, el título y enlaces a archivos externos.

• **meta charset="UTF-8"**: permite mostrar correctamente caracteres como ñ y vocales acentuadas.

• **meta viewport**: ayuda a adaptar la página a pantallas móviles.

• **<title>**: título que aparece en la pestaña del navegador.

• **<link>**: conecta la hoja de estilos CSS.

• **<script>**: carga JavaScript. El atributo **defer** hace que se ejecute después de que el HTML haya sido analizado.

• **<body>**: contiene el contenido visible de la página.

2.2. Etiquetas HTML comunes

Etiqueta	Función	Ejemplo
<h1>	Título principal.	Bienvenidos
<h2>	Título de sección.	Servicios
	Párrafo.	Un texto.
<a>	Enlace.	Contacto
<img>	Imagen.	<img src="foto.jpg" alt="Descripción">
	Contenedor genérico.	...
<section>	Sección temática.	<section>...</section>
<header>	Cabecera.	<header>...</header>
<nav>	Navegación.	<nav>...</nav>
<main>	Contenido principal.	<main>...</main>
<footer>	Pie de página.	<footer>...</footer>
<button>	Botón.	<button>Enviar</button>
<form>	Formulario.	<form>...</form>
<input>	Campo de entrada.	<input type="text">

2.3. Atributos, id y class

Los atributos aportan información adicional a un elemento. En este ejemplo, **href** indica el destino del enlace:

```
<a href="https://www.google.com">Ir a Google</a>
```

En este botón, **id** identifica un elemento concreto y **class** asigna una clase que puede reutilizarse:

```
<button id="botonEnviar" class="boton">Enviar</button>
```

En CSS, las clases se seleccionan con un punto (**.boton**) y los ID con el símbolo numeral (**#botonEnviar**). En HTML se escriben sin esos símbolos.

3. CSS: el diseño visual

CSS significa *Cascading Style Sheets*. Define cómo se presentan visualmente los elementos HTML.

3.1. Estructura de una regla CSS

```
selector {
    propiedad: valor;
}
```

En **h1 { color: red; }**, **h1** es el selector, **color** es la propiedad y **red** es el valor. Las llaves delimitan las declaraciones y el punto y coma termina una declaración.

3.2. Ejemplo de CSS

```
/* Estilos generales */
body {
    font-family: Arial, sans-serif;
    background-color: #f5f5f5;
    color: #212121;
}

/* Título principal */
h1 {
    color: #0097A7;
    text-align: center;
}

/* Botón */
.boton {
    background-color: #0097A7;
    color: white;
    padding: 12px 24px;
    border: none;
    border-radius: 8px;
    cursor: pointer;
}
```

3.3. Selectores comunes

Selector	Qué selecciona	Ejemplo
h1	Todos los elementos h1.	h1 { color: red; }
.boton	Elementos con clase boton.	.boton { padding: 10px; }
#principal	Elemento con ID principal.	#principal { background: white; }
*	Todos los elementos.	* { box-sizing: border-box; }
body p	Párrafos descendientes de body.	body p { color: gray; }
button:hover	Botones al pasar el cursor.	button:hover { opacity: .8; }

Ejemplo de conexión entre HTML y CSS:

```
<!-- HTML -->
<p class="descripcion">Una cerveza para disfrutar.</p>

/* CSS */
.descripcion {
    color: gray;
}
```

3.4. Propiedades CSS para comenzar

Propiedad	Función	Ejemplo
color	Color del texto.	color: white;
background-color	Color de fondo.	background-color: black;
font-family	Familia tipográfica.	font-family: Arial, sans-serif;
font-size	Tamaño de texto.	font-size: 2rem;
font-weight	Grosor de la tipografía.	font-weight: 700;
text-align	Alineación del texto.	text-align: center;
width / max-width	Ancho / ancho máximo.	max-width: 1200px;
height	Alto.	height: 200px;
margin	Espacio exterior.	margin: 20px;
padding	Espacio interior.	padding: 20px;
border	Borde.	border: 1px solid black;
border-radius	Esquinas redondeadas.	border-radius: 10px;
display	Modo de presentación.	display: flex;
gap	Espacio entre elementos flex/grid.	gap: 20px;
position	Método de posicionamiento.	position: relative;

3.5. Box model: el modelo de caja

Cada elemento puede entenderse como una caja con cuatro partes: contenido (content), espacio interior (padding), borde (border) y espacio exterior (margin).

```
.tarjeta {
    width: 300px;
    padding: 20px;
    border: 2px solid black;
    margin: 20px;
}
```

Con el modelo predeterminado, el ancho total sin márgenes es 344 px: 300 de contenido + 40 de padding + 4 de borde. Por eso suele utilizarse **box-sizing: border-box**, que incluye padding y borde dentro del ancho declarado.

```
* {
    box-sizing: border-box;
}
```

3.6. Flexbox y Grid

Flexbox distribuye elementos principalmente en una dimensión (fila o columna):

```
.contenedor {
    display: flex;
    justify-content: center;
    align-items: center;
    gap: 20px;
}
```

justify-content distribuye los elementos en el eje principal; **align-items** los alinea en el eje transversal; **gap** crea espacio entre ellos.

CSS Grid permite organizar filas y columnas:

```
.galeria {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
}
```

Ruta sugerida: primero aprendé Flexbox y después incorporá Grid.

4. JavaScript: lógica e interactividad

JavaScript es un lenguaje de programación. Permite trabajar con datos, realizar cálculos, tomar decisiones y responder a acciones del usuario. Se utiliza para abrir menús, validar formularios, actualizar contadores, filtrar productos, modificar contenido y comunicarse con servidores.

4.1. Variables: let, const y var

```
let nombre = "Bruma";
let precio = 5000;
const iva = 0.21;

let contador = 0;
contador = 1;
contador = 2;

const marca = "Bruma";
```

let permite reasignar el valor; **const** impide reasignar la variable; **var** es una forma antigua de declaración y normalmente conviene evitarla al empezar.

Importante: **const** no vuelve inmutable a un objeto. Impide reasignar la variable, pero las propiedades del objeto pueden cambiar.

4.2. Tipos de datos

```
let nombre = "Andrés";       // String: texto
let edad = 25;               // Number: número
let activo = true;           // Boolean: verdadero/falso
let resultado = null;        // Null: ausencia intencional de valor
let sinDefinir;              // Undefined: no se asignó valor

const bebidas = ["IPA", "Golden", "Stout"];

const producto = {
    nombre: "Cerveza artesanal",
    precio: 5000,
    disponible: true
};

console.log(producto.nombre);
console.log(producto.precio);
```

Un **array** es una colección ordenada de elementos. Un **objeto** agrupa propiedades y valores. **console.log()** muestra información en la consola del navegador, útil para aprender y depurar.

4.3. Operadores

```
let suma = 10 + 5;
let resta = 10 - 5;
let multiplicacion = 10 * 5;
let division = 10 / 5;

console.log(10 > 5);     // true
console.log(10 < 5);     // false
console.log(10 === 10);  // true
console.log(10 !== 5);   // true
```

Como regla general, al comenzar utilizá **===** y **!==** para comparar valores sin conversiones de tipo automáticas.

```
let tieneEntrada = true;
let esMayor = true;

console.log(tieneEntrada && esMayor);
```

Operadores lógicos: **&&** (AND: ambas condiciones verdaderas), **||** (OR: al menos una verdadera) y **!** (NOT: invierte el valor lógico).

4.4. Condicionales

```
let edad = 20;

if (edad >= 18) {
    console.log("Es mayor de edad");
} else {
    console.log("Es menor de edad");
}
```

JavaScript evalúa la condición. Si es verdadera, ejecuta el primer bloque; si es falsa, ejecuta el bloque **else**. También existe **else if** para evaluar condiciones adicionales.

4.5. Funciones

```
function saludar() {
    console.log("¡Hola!");
}

saludar();

function saludarA(nombre) {
    console.log("Hola, " + nombre);
}

saludarA("Andrés");
saludarA("María");

function sumar(a, b) {
    return a + b;
}

const resultado = sumar(10, 5);
console.log(resultado);
```

Una función es un bloque reutilizable. Los parámetros reciben datos; los argumentos son los valores que se pasan al llamar la función; **return** devuelve un resultado.

4.6. Bucles

```
for (let i = 0; i < 5; i++) {
    console.log(i);
}

const bebidas = ["IPA", "Golden", "Stout"];

for (const bebida of bebidas) {
    console.log(bebida);
}
```

En el primer bucle, **let i = 0** inicializa el contador, **i < 5** es la condición para continuar y **i++** incrementa el contador. **for...of** recorre los elementos de una colección iterable, como un array.

5. Conectar JavaScript con HTML: el DOM

DOM significa *Document Object Model*. Es la representación del documento HTML que el navegador expone como objetos, de modo que JavaScript pueda consultar y modificar elementos.

```
<h1 id="titulo">Bienvenidos</h1>
<button id="boton">Cambiar título</button>

const titulo = document.querySelector("#titulo");
const boton = document.querySelector("#boton");

titulo.textContent = "¡Hola, mundo!";
```

document representa el documento; **querySelector()** busca el primer elemento que coincide con un selector CSS; **textContent** permite consultar o modificar el texto del elemento.

5.1. Responder a un clic con un evento

```
const titulo = document.querySelector("#titulo");
const boton = document.querySelector("#boton");

boton.addEventListener("click", function () {
    titulo.textContent = "¡Hola, mundo!";
});
```

addEventListener() registra una función para un evento. **click** es el evento; la función dentro del método se ejecuta cuando se hace clic. En este caso, cambia el texto del título.

Idea clave: HTML crea el botón, JavaScript detecta el clic y actualiza el contenido.

6. Conectar JavaScript con CSS

Una forma habitual de cambiar la apariencia desde JavaScript es agregar o quitar clases CSS.

```
<!-- HTML -->
<p id="mensaje">Estoy aprendiendo programación.</p>
<button id="boton">Cambiar diseño</button>

/* CSS */
.destacado {
    color: white;
    background-color: #0097A7;
    padding: 20px;
    border-radius: 10px;
}

/* JavaScript */
const mensaje = document.querySelector("#mensaje");
const boton = document.querySelector("#boton");

boton.addEventListener("click", function () {
    mensaje.classList.toggle("destacado");
});
```

classList.toggle() agrega la clase si no está presente y la quita si ya existe. Cuando la clase está presente, CSS aplica sus reglas.

Consejo: para cambios visuales habituales, es preferible definir clases en CSS y activarlas desde JavaScript en lugar de escribir todos los estilos directamente desde JavaScript.

7. Proyecto completo: HTML + CSS + JavaScript

Creá una carpeta llamada **mi-proyecto** con esta estructura. Las rutas deben coincidir exactamente con los nombres y las carpetas.

```
mi-proyecto/
■■■ index.html
■■■ css/
■   ■■■ styles.css
■■■ js/
    ■■■ script.js
```

Archivo 1: index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mi primer proyecto</title>
    <link rel="stylesheet" href="css/styles.css">
    <script src="js/script.js" defer></script>
</head>
<body>
    <main class="contenedor">
        <h1 id="titulo">Mi primera página interactiva</h1>
        <p id="mensaje">Estoy aprendiendo HTML, CSS y JavaScript.</p>
        <button id="boton">Cambiar mensaje</button>
    </main>
</body>
</html>
```

La etiqueta **link** conecta HTML con CSS. La etiqueta **script** conecta HTML con JavaScript. Las rutas son relativas a la ubicación de index.html.

Archivo 2: css/styles.css

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    min-height: 100vh;
    font-family: Arial, sans-serif;
    background-color: #f5f5f5;
    color: #212121;
    display: flex;
    justify-content: center;
    align-items: center;
}

.contenedor {
    width: 90%;
    max-width: 600px;
    padding: 40px;
    background-color: white;
    border-radius: 12px;
    text-align: center;
    box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}

h1 { color: #0097A7; }
p { line-height: 1.6; }

button {
    padding: 12px 20px;
    background-color: #0097A7;
    color: white;
    border: none;
    border-radius: 6px;
    cursor: pointer;
}

button:hover { background-color: #006874; }

.destacado {
    padding: 15px;
    color: white;
    background-color: #0097A7;
    border-radius: 6px;
}
```

Archivo 3: js/script.js

```
const boton = document.querySelector("#boton");
const titulo = document.querySelector("#titulo");
const mensaje = document.querySelector("#mensaje");

boton.addEventListener("click", function () {
    mensaje.classList.toggle("destacado");

    if (mensaje.classList.contains("destacado")) {
        titulo.textContent = "¡Excelente trabajo!";
        mensaje.textContent = "Ya conectaste HTML, CSS y JavaScript.";
        boton.textContent = "Restablecer";
    } else {
        titulo.textContent = "Mi primera página interactiva";
        mensaje.textContent = "Estoy aprendiendo HTML, CSS y JavaScript.";
        boton.textContent = "Cambiar mensaje";
    }
});
```

Qué hace el proyecto

• HTML establece la estructura.

• CSS define el diseño.

• JavaScript detecta el clic.

• JavaScript actualiza los textos.

• JavaScript agrega o quita la clase destacado.

• CSS cambia la apariencia cuando la clase está presente.

classList.contains() comprueba si un elemento tiene la clase indicada. El condicional usa ese resultado para elegir qué textos mostrar.

8. Glosario rápido

Concepto	Tecnología	Significado
Etiqueta	HTML	Marca que define un elemento, como <p>.
Elemento	HTML	La etiqueta y su contenido, cuando corresponde.
Atributo	HTML	Información adicional, como href, id o class.
Selector	CSS	Indica qué elementos reciben los estilos.
Propiedad	CSS	Característica visual que se modifica.
Valor	CSS	Valor asignado a una propiedad.
Variable	JavaScript	Nombre asociado a un valor.
Función	JavaScript	Bloque de código reutilizable.
Condicional	JavaScript	Ejecuta código según una condición.
Evento	JavaScript	Algo que sucede, como un clic.
DOM	Navegador / JS	Representación del documento que puede consultarse y modificarse.
Método	JavaScript	Funcionalidad asociada a un objeto.
Array	JavaScript	Colección ordenada de elementos.
Objeto	JavaScript	Estructura que agrupa propiedades y valores.
Depuración	Desarrollo	Proceso de localizar y corregir errores.
Responsive	CSS / diseño	Adaptación de la interfaz a distintas pantallas.

9. Ruta de aprendizaje recomendada

Etapa 1: HTML

Estudiá estructura básica, etiquetas, atributos, enlaces, imágenes, listas, formularios y elementos semánticos. **Objetivo:** construir la estructura de una página sin depender constantemente de un tutorial.

Etapa 2: CSS

Estudiá selectores, especificidad, colores, tipografía, box model, márgenes, padding, Flexbox, Grid, pseudoclases, media queries y diseño responsive. **Objetivo:** convertir una estructura sencilla en una página visualmente atractiva.

Etapa 3: JavaScript

Estudiá variables, tipos de datos, operadores, condicionales, bucles, funciones, arrays, objetos, DOM y eventos. **Objetivo:** comprender cómo crear interacciones y resolver problemas mediante código.

Etapa 4: Integración

Practicá con tarjetas de producto, galerías, menús desplegables, listas de tareas y landing pages. Más adelante podés sumar Git y GitHub.

10. Hoja de referencia rápida

Si querés...	Usá...
Crear un título	HTML: <h1>
Insertar una imagen	HTML: <img>
Crear un botón	HTML: <button>
Cambiar el color del texto	CSS: color
Cambiar el fondo	CSS: background-color
Alinear elementos en una fila o columna	CSS: Flexbox
Crear columnas	CSS: Grid
Guardar un dato	JavaScript: variable
Tomar una decisión	JavaScript: if
Repetir instrucciones	JavaScript: bucle
Reutilizar instrucciones	JavaScript: función
Detectar un clic	JavaScript: evento
Cambiar un texto	JavaScript: textContent
Agregar o quitar una clase	JavaScript: classList
Mostrar datos para depurar	JavaScript: console.log()

11. Cómo practicar sin depender de la IA

• Intentá resolver el problema por tu cuenta durante unos minutos antes de pedir ayuda.

• Pedí explicaciones, no solo soluciones: preguntá por qué ocurre un error.

• No copies código que no entendés; probá cambiar valores y observá el resultado.

• Usá la IA para revisar el código, encontrar errores y sugerir mejoras; verificá los cambios en el navegador.

• Practicá también sin IA para comprobar qué conceptos podés aplicar de forma independiente.

La IA puede acelerar el aprendizaje, pero el objetivo es que puedas comprender, evaluar y modificar el código.

12. Actividad práctica

• **Paso 1:** escribí el HTML y comprobá que aparezcan el título, el párrafo y el botón.

• **Paso 2:** conectá CSS y ajustá colores, espacios, tipografía y alineación.

• **Paso 3:** conectá JavaScript y hacé que el botón cambie el texto.

• **Paso 4:** agregá y quitá la clase destacado al hacer clic.

• **Paso 5:** cambiá los textos, los colores y los tamaños para personalizar el proyecto.

• **Paso 6:** probá la página en una ventana estrecha y ajustá el diseño para celulares.

Preguntas para comprobar que entendiste:

• ¿Qué ocurre si el nombre o la ruta del archivo CSS es incorrecto?

• ¿Cuál es la diferencia entre class e id?

• ¿Qué diferencia hay entre let y const?

• ¿Qué hace addEventListener?

• ¿Cómo puede JavaScript cambiar la apariencia sin escribir estilos directamente?

Si podés responder estas preguntas con tus propias palabras y modificar el proyecto sin copiar cada línea, estás construyendo una base útil.

Recordatorio final: HTML construye · CSS diseña · JavaScript hace que las cosas sucedan. Aprendé por etapas, practicá con proyectos pequeños y volvé a esta guía cada vez que necesites consultar un concepto.